Bert Model

Monday, March 23, 2026 11:07 PM

Feature	BERT	GPT (LLMs)
Developed by	Google	OpenAI
Architecture	**Encoder-only Transformer**	**Decoder-only Transformer**
Context understanding	**Bidirectional (← →)**	**Unidirectional (left → right)**
Training Objective	Masked Language Model (MLM)	Next Word Prediction
Use case	Understanding tasks	Generation tasks
Example	Classification	Chatbots, text generation

Bidirectional meaning in BERT

- BERT reads a word using both left and right context simultaneously.

Example (Very Important)

Sentence:

He sat on the bank of the river
Focus word: **bank**

How BERT understands:

- Left context → *He sat on the*
- Right context → *of the river*

So BERT concludes:

bank = river side (not financial bank)

BERT processes:
← full context + word + full context →
So meaning is **context-aware and accurate**

Compare with Unidirectional Model

If model reads only left → right:

He sat on the bank
It may think:

bank can be misunderstood as financial institution
Because it **has not seen "river" yet**

When to Use **BERT** vs **GPT (LLMs)**

Typical Tasks

Use BERT when you need:

- **Text Classification**

    - Spam detection
    - Sentiment analysis
- **Named Entity Recognition (NER)**
    - Identify names, locations, dates
- **Extractive Question Answering**
    - Answer comes **from given passage**

Real Rule: If output is:

- **Label / Score / Span / Decision**
  Use **BERT**

Use **GPT (LLMs)** when your goal is **GENERATING text**

Typical Tasks

- Chatbots (like ChatGPT)
- Content writing
- Code generation
- Summarization
- Explanation / reasoning

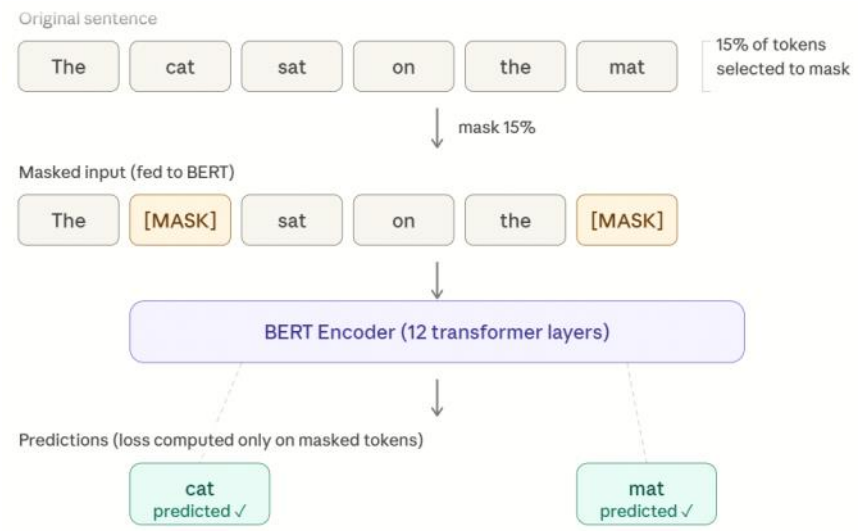
Scenario	Use
Classify email as spam/not spam	BERT
Extract answer from paragraph	BERT
Match resume with job description	BERT
Generate answer/explanation	GPT
Write code / essay	GPT

Masked Language Modeling (MLM) in BERT

MLM is one of BERT's two core pre-training objectives (the other being Next Sentence Prediction). The idea is elegantly simple: **hide some words in a sentence, and make the model predict what they were** — forcing it to learn deep contextual understanding from both left and right context simultaneously.

This is what made BERT revolutionary. Earlier models like GPT were unidirectional (left-to-right). BERT reads the entire sentence at once with words masked out, making it truly **bidirectional**.

Here's how the masking strategy works in practice:



The **Masked Language Model (MLM)** is the "secret sauce" that allows BERT (Bidirectional Encoder Representations from Transformers) to understand context from both sides of a word simultaneously. Before BERT, models usually read text left-to-right or right-to-left. BERT's MLM approach allows it to look at the entire sentence at once, much like how you might solve a "fill-in-the-blank" puzzle.

How MLM Works: Step-by-Step

To train BERT, researchers take a massive corpus of text and "corrupt" it before feeding it into the model. Here is the process:

1. Tokenization and Masking

BERT takes a sentence and breaks it into tokens. It then randomly selects **15%** of those tokens to be processed. Of that 15%:

- **80%** are replaced with a [MASK] token.
- **10%** are replaced with a random word (to keep the model from getting too cozy with just the mask).
- **10%** are left unchanged (to ensure the model maintains a bias toward the actual observed word).

2. The Contextual "Look-Around"

Unlike older models that would try to predict the next word based only on previous ones, BERT uses the **Transformer Architecture** to look at every other word in the sentence to figure out what belongs in the blank.

3. Prediction and Loss

The model outputs a probability distribution for the masked position. It compares its guess against the actual original word. If it guessed "cat" but the word was "dog," the model adjusts its internal weights to be more accurate next time.

A Concrete Example

Let's look at the sentence: **"The chef prepared a delicious meal for the guests."**

Step	Action	Resulting Sentence
Input	Original Text	"The chef prepared a delicious meal for the guests."
Masking	Replace "delicious"	"The chef prepared a **[MASK]** meal for the guests."
Processing	BERT analyzes context	It sees "chef," "prepared," and "meal."
Prediction	Output Probabilities	"delicious" (92%), "tasty" (5%), "bad" (1%)

Question 1 (MLM – Numerical)

Consider the sentence:

I love [MASK] intelligence
After passing through BERT encoder, the hidden vector for [MASK] is:

Mask_vector = [0.6, -0.2, 0.5]
Vocabulary:

Word	Vector
artificial	[0.5, 0.1, 0.4]
natural	[0.3, -0.2, 0.6]
machine	[0.4, 0.3, 0.2]

Tasks:

1. Compute logits using:

$$logit = h_{mask} \cdot W^T$$

1. Apply **softmax**
2. Find predicted word

Question (MLM with TWO Masks – Numerical)

Given Sentence:

I love [MASK1] and [MASK2]

After BERT Encoder:

Hidden vectors:

h_mask1 = [0.5, -0.1, 0.6]
h_mask2 = [0.2, 0.4, -0.3]

Vocabulary:

Word	Vector
apples	[0.4, 0.2, 0.5]
bananas	[0.3, -0.3, 0.7]
mangoes	[0.6, 0.1, 0.2]

Predict:

- Best word for **MASK1**
- Best word for **MASK2**

Next Sentence Prediction (NSP) in BERT

Next Sentence Prediction (NSP) is a **pre-training task** in BERT where the model learns:
"Does sentence B logically come after sentence A?"

BERT is not only about word meaning—it also learns:

- Sentence relationships
- Logical flow
- Context across sentences

✓ Useful for:

- Question Answering
- Natural Language Inference
- Dialogue systems

How Training Data is Prepared

For each pair of sentences:

50% cases (Positive)

Sentence B is the **actual next sentence**

A: I am feeling hungry.
B: I will order food.
→ Label = IsNext (1)

50% cases (Negative)

Sentence B is **random**

A: I am feeling hungry.
B: The car is red.
→ Label = NotNext (0)

Input Format in BERT

Two sentences are combined as:
[CLS] Sentence A [SEP] Sentence B [SEP]

Example

[CLS] I love AI [SEP] It is powerful [SEP]

Processing Through BERT

- Entire input passes through **Transformer encoder**
- Self-attention connects:
  - Words in A ↔ Words in B

Model understands **relationship between sentences**

Role of [CLS] Token (Very Important)

After encoding:

h_cls = representation of both sentences
[CLS] acts as a **summary vector**

Prediction Step

The [CLS] vector is passed to:

Linear Layer → Softmax

Output Example

IsNext → 0.91
NotNext → 0.09
Final Prediction:IsNext

Complete Flow

[CLS] A [SEP] B [SEP]
↓
Encoder (Self-Attention)
↓
[CLS] vector
↓
Softmax
↓
IsNext / NotNext